Information Modeling Using EXPRESS — An Introduction

Information?

Definition

Information is knowledge of ideas, facts and/or processes.

- Information can be exchanged between partners.
- Exchange may be real time or delayed.
- Information storage is a special case of communication.

Information Model?

Definition

An information model is a formal description of types (classes) of ideas, facts and processes which together form a model of a portion of interest of the real world.

- If a model is written in EXPRESS or any other computer sensible representation, it has the additional property of being *computer processible*.
- An information model may be *instantiated* or *populated* to represent **particular** ideas, facts and processes.
- An information model that is specialized to take account of a particular instantiation method is a *concrete* model. One that is implementation independent is a *conceptual* model.

Why Formal Models?

- Time flies like an arrow.
- Fruit flies like a banana.
- Joe saw the man with the telescope.
- Braces

  UK

  Hold up one’s trousers.

  USA

  Adjust one’s teeth.

- The date 1-3-91

UK

1st March 1991 (or is it 1891?)

USA

3rd January 1991

Information Model

Purpose

To identify clearly the objects in a 'Universe of Discourse' in order to enable people (and potentially computer systems) to communicate precisely about a domain of common interest.

Comprises

An information model is composed of

- objects
- relationships
- constraints

which, taken together, provide a complete and unambiguous formal representation of a Universe of Discourse.

IM is NOT

An Information Model is

- NOT a database definition (even though terms such as *schema* are common.
- NOT a data structure definition (even though data instances of the model could be structured)
- NOT a program (even though procedural code and algorithms may be in the model)

A populated instance of an IM may be maintained using DB or similar technologies. IM constraints are often implemented via programatic code.

IM Description Methods

An Information Model may be described:

Textually

using a formally defined lexical language. Examples include EXPRESS, IISyCL (Integrated Information Systems Constraint Language), VDM (Vienna Development Method), etc.

Graphically

using an iconic or diagramatic language such as EXPRESS-G, IDEF1X, OMT, UML, etc.

NOTE	Supplementing textual models with diagrams can help the reader's understanding.

> Graphical models nearly always require supplemental text for completeness.

EXPRESS Development

EXPRESS developed as an information modeling language to meet the needs of product data exchange model definition.

- First version, called DSL, developed under the USAF funded PDDI program (early '80s).
- PDES reviewed NIAM and IDEF1X. Neither had the power needed.
- PDES started extending EXPRESS.
- STEP mandated all ‘Normative’ models to be in EXPRESS.
- Language still evolving.

REVIEWS

The language is subject to ongoing review within STEP and by other users. Also international public review as part of ISO standardization:

Early 1989
: ISO Draft Proposal ballot

Mid 1991
: ISO Committee Draft ballot

Oct 1991
: Ballot successful — Draft International Standard status.

Mid 1993
: Approved for registration as an International Standard (ISO 10303 Part 11).

End 1994
: Published as International Standard ISO 10303-11:1994.

End 2003
: Edition 2 approved as an International Standard.

Table 1. LANGUAGE COMPARISON

Characteristic	Others	EXPRESS
Modeling		
Form	Graphics	Programmatic
Flavor	Relational	OO
Objects	X	X
Relationships	X	X

Attributes	X	X
Derived Atts.		X
Domain	Entity	Entity + Type
Sequencing		X
Cardinalities	Limited	Any
Constraints		
Domain	Limited	Any
Roles	Limited	Any
Categorization	Limited	Broad
Miscellaneous		
Multi-page	Some	X
Algorithms		X
Scoping		X

Graphical Models

- Excellent for group explanations and work.
- Easy to follow (but can take a lot of wall space).
- Model development may be superficial (it looks right).
- Some drawing tools may exist, or can use CAD system.
- Effectively, not computer processible (What You See Is All You’ve Got).

Textual Models

- Good formal definition or mathematical support.
- May be non-intuitive (e.g logic based methods).
- Complex constraints and rules.
- Computer processible.
- Syntax and semantic checking.
- Potential for automatic implementation (for model simulation and test).

EXPRESS is:

- A language family for representing an information model.
- Computer processible.
- Under development since early '80s.

- Superset of NIAM and IDEF1X representation capabilities.
- Exhibits an object oriented flavor.
- Been an ISO standard since 1994 (2nd Edition 2004)
- Has several aspects (subsets)

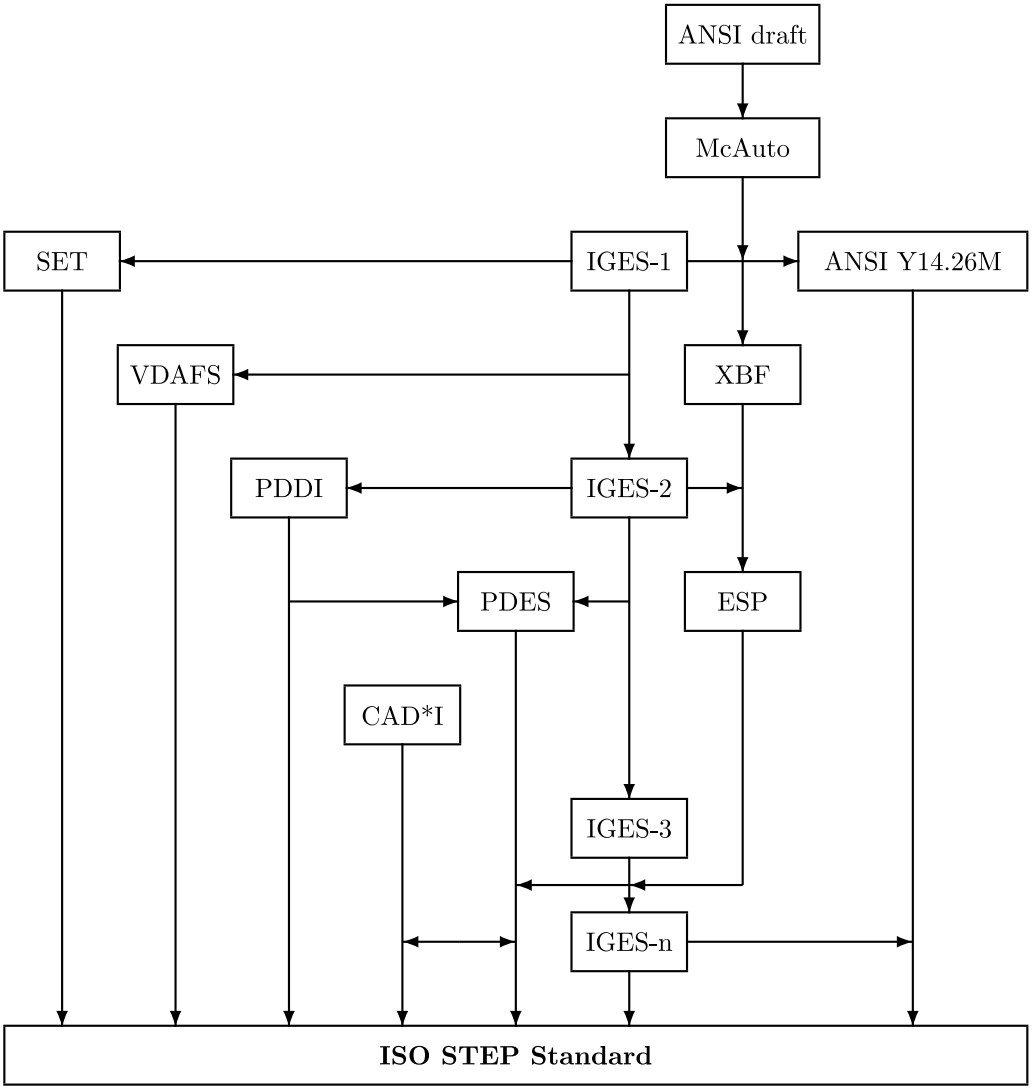
EXPRESS Aspects

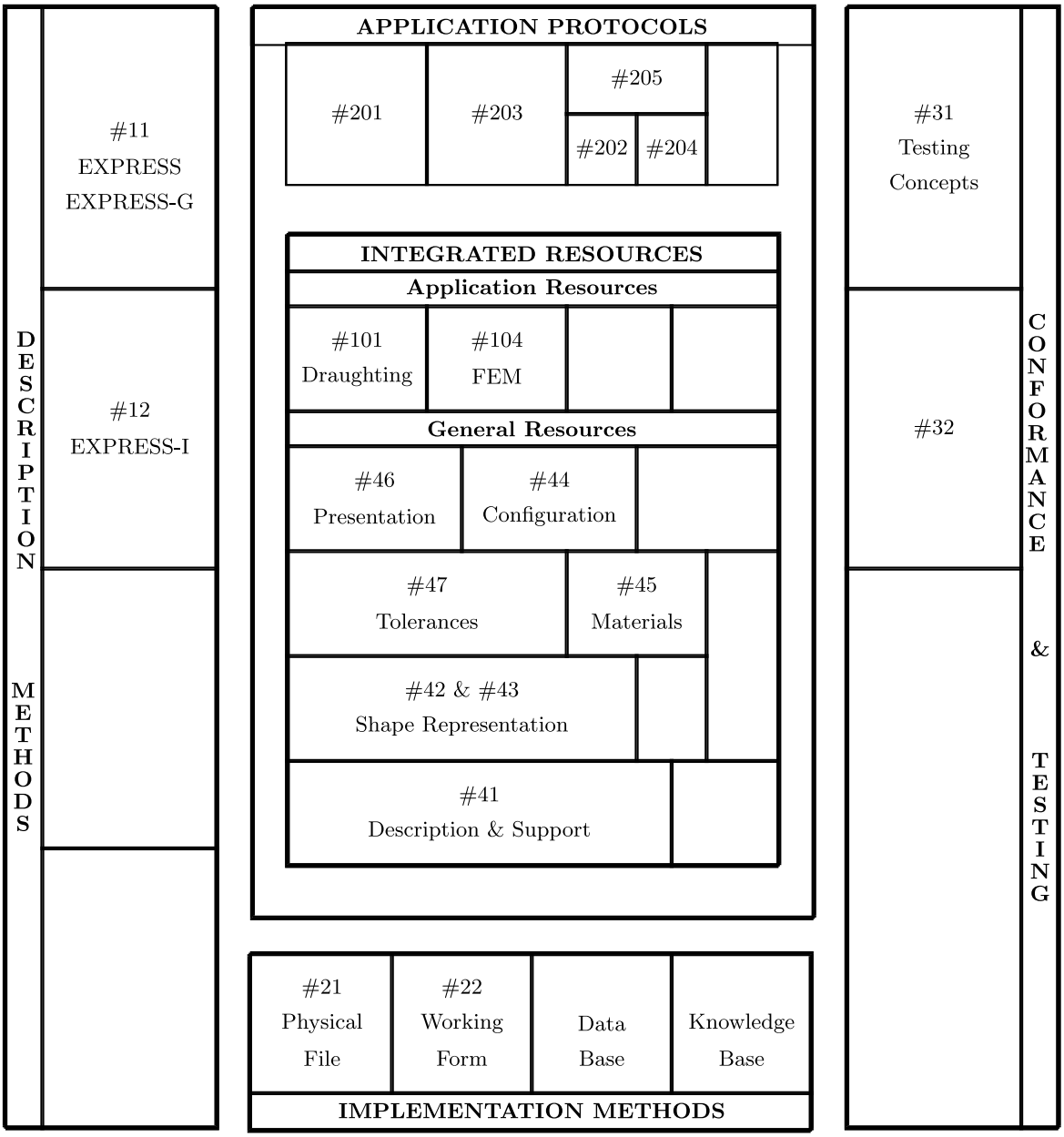
- Textual language.
- Modeling of things and relationships (implementation independent).
- Algorithms for arbitrary constraint specifications.
- Modeling of implementation dependent data structures.
- Graphical form as a subset of textual form (EXPRESS-G).
- An ‘instantiation’ format (EXPRESS-I).
- Transformation specification (EXPRESS-X).

EXPRESS Usage

- Definition of the STEP models (200+ people from 20+ countries)
- Reverse engineering of a DBMS system
- Software Specification Document for a CAD geometry processor
- Electronic standards (VHDL, EDIF, CFI etc)
- Many European ESPRIT projects
- Data Definition Language for OO Database
- Geological modeling
- Genome modeling

Other uses are possible, such as using EXPRESS to define the syntax, grammer, and semantics of the EXPRESS language.


ANSI draft
McAuto
SET
IGES-1
ANSI Y14.26M
VDAFS
XBF
PDDI
IGES-2
PDES
ESP
CAD*I
IGES-3
IGES-n
ISO STEP Standard


DESCRIPTION METHODS
#11 EXPRESS EXPRESS-G
#12 EXPRESS-I
APPLICATION PROTOCOLS
#201
#203
#205
#202
#204
INTEGRATED RESOURCES
Application Resources
#101 Draughting
#104 FEM
General Resources
#46 Presentation
#44 Configuration
#47 Tolerances
#45 Materials
#42 & #43 Shape Representation
#41 Description & Support
#21 Physical File
#22 Working Form
Data Base
Knowledge Base
IMPLEMENTATION METHODS
#31 Testing Concepts
#32
CONFORMANCE & TESTING

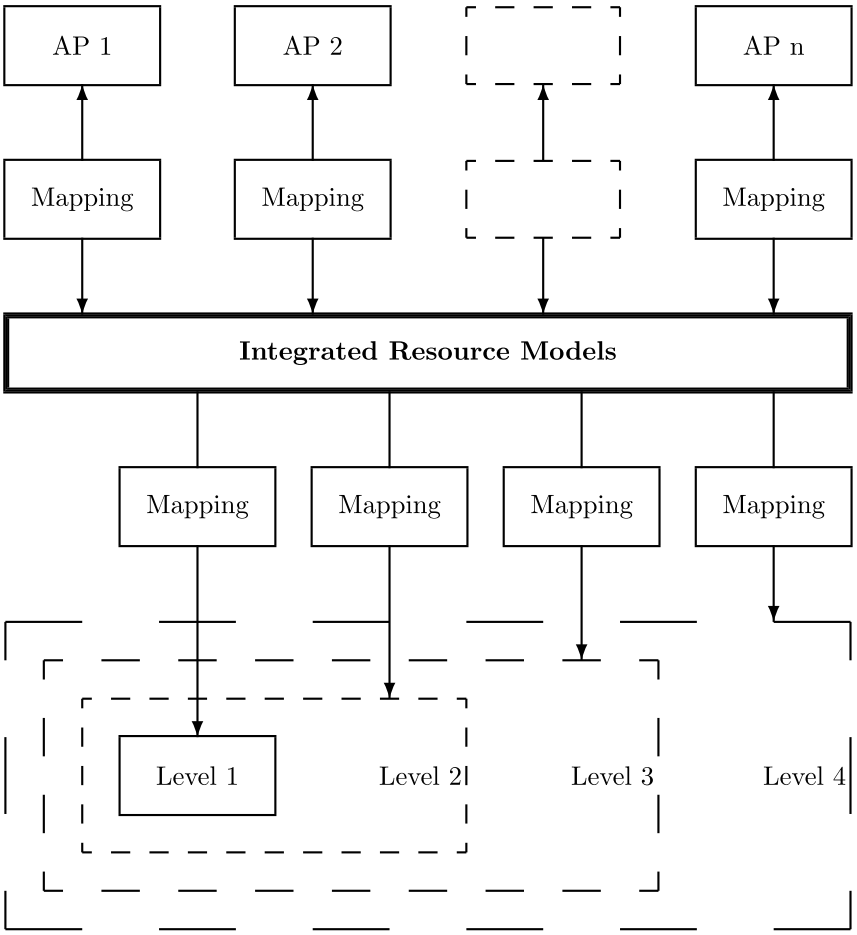


Figure 1. STEP ARCHITECTURE

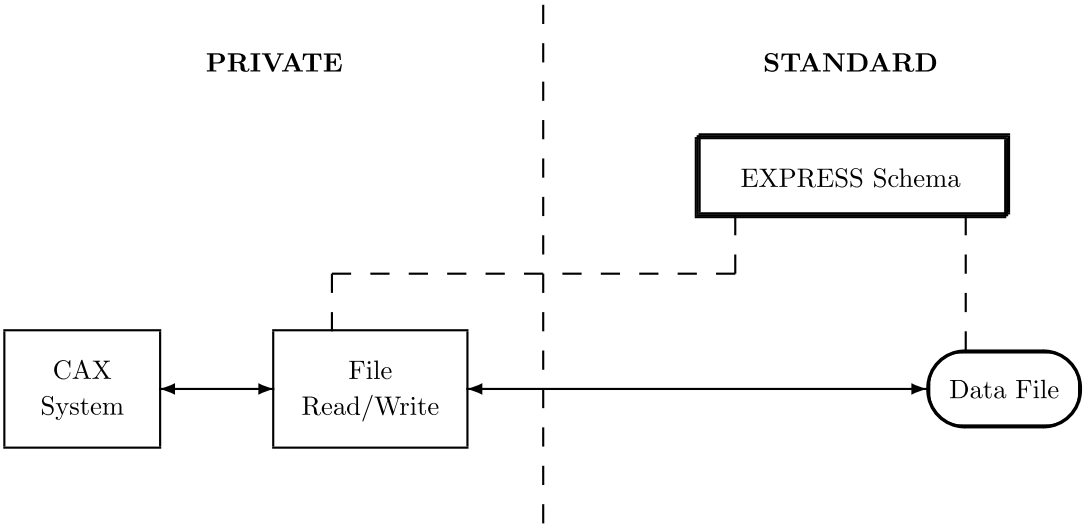


Figure 2. Level 1 Exchange

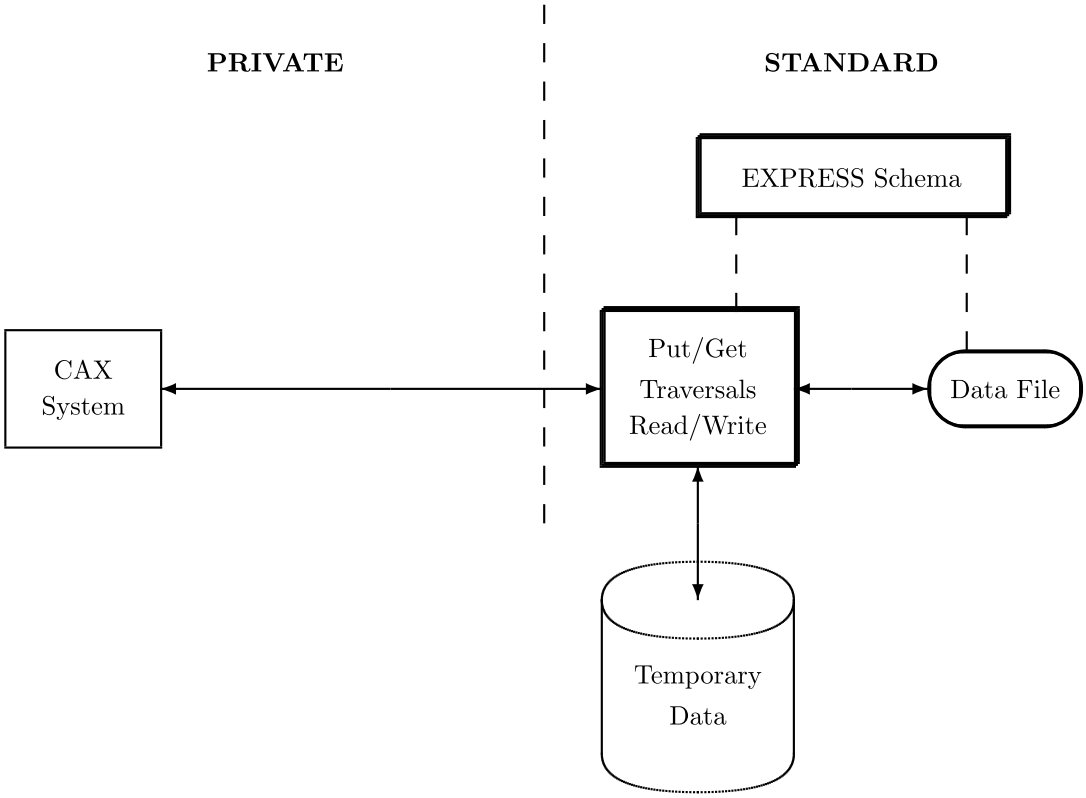


Figure 3. Level 2 Exchange

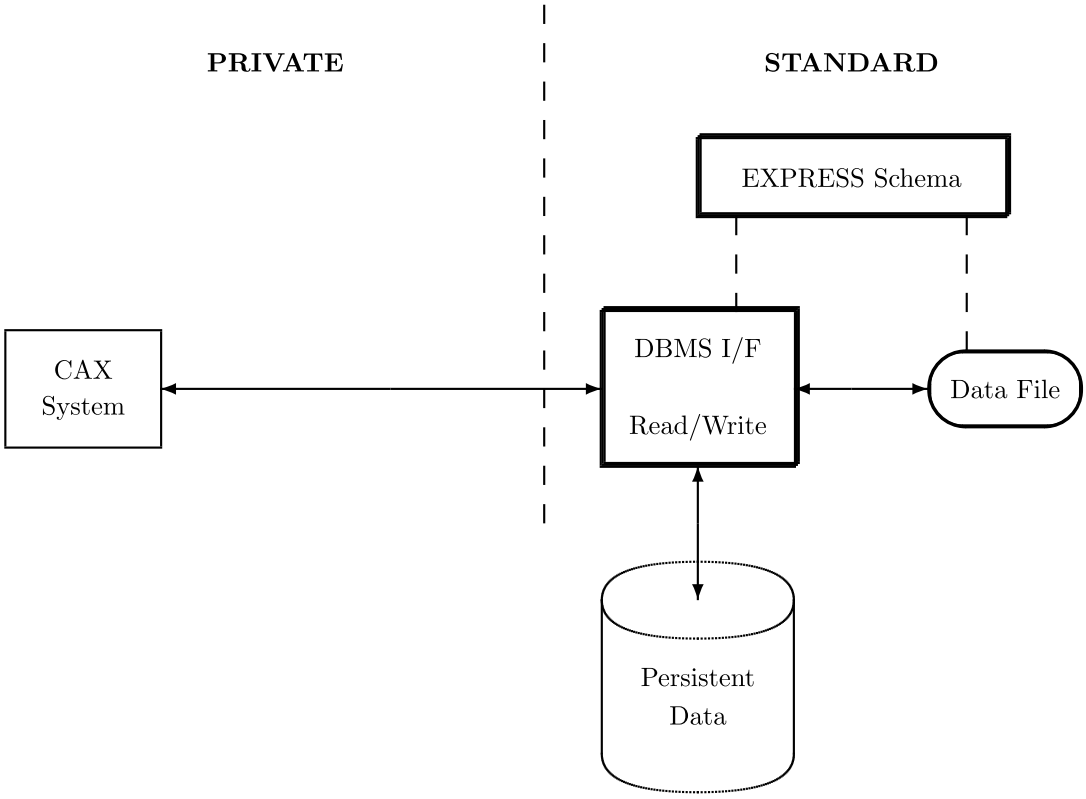


Figure 4. LEVEL 3 EXCHANGE

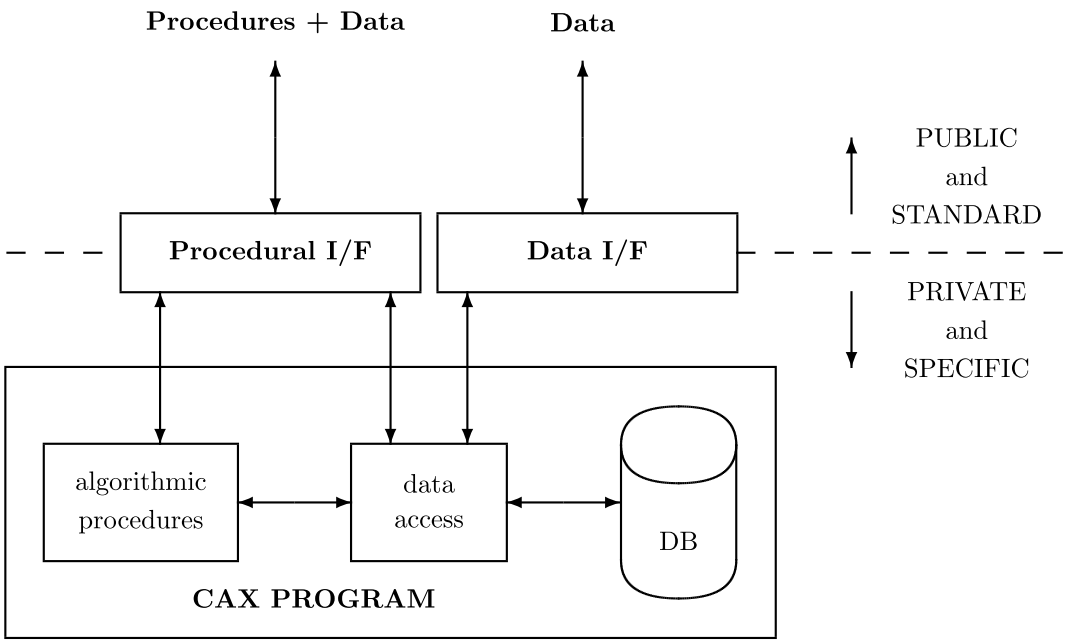


Figure 5. Procedural Exchange

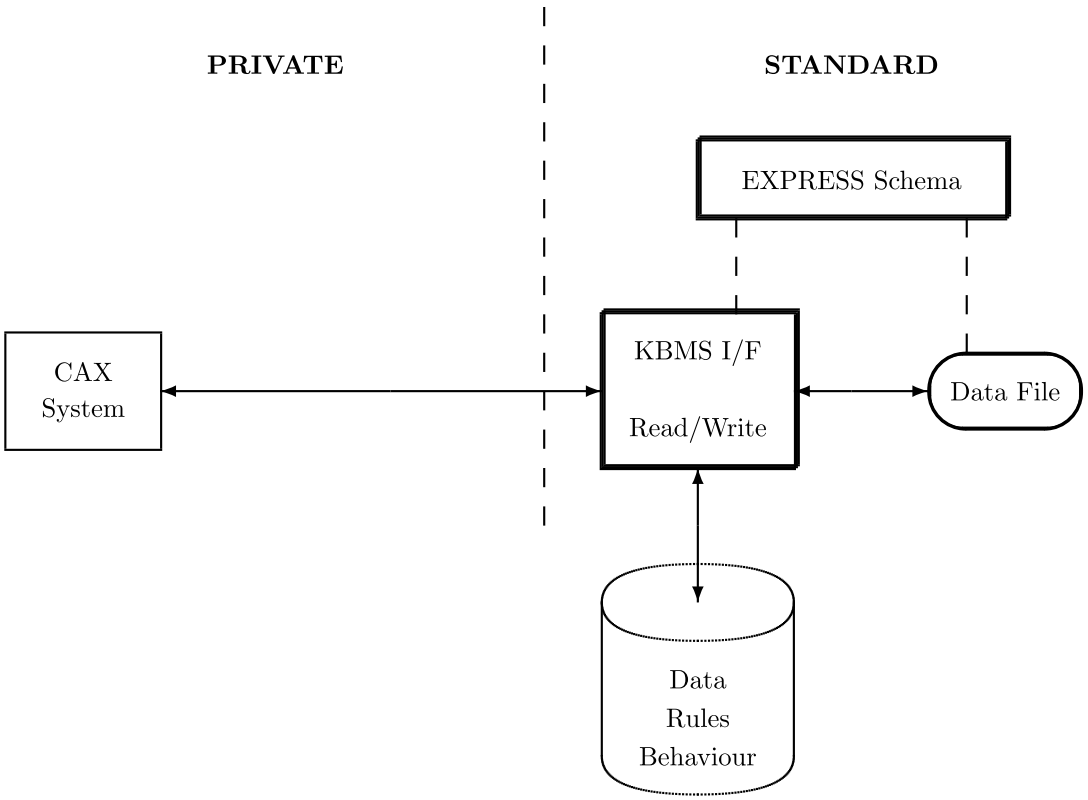


Figure 6. LEVEL 4 EXCHANGE

EXPRESS Primitives

- Number, Integer, Real, Binary, String, Boolean (T/F), Logical (T/F/U)
- Schema, Entity, Rule, Function, Procedure, Type (Defined, Select, Enumeration)
- Aggregations — Array, Set, List, Bag
- Pascal-like procedural language

Simple Types

- `n : NUMBER` which has 'subtypes'
  - `i : INTEGER`
  - `r : REAL`

These types may be given a 'precision'. E.g `REAL(6)`

Various operations such as `+, -, //, ">="`, etc. may be applied to these types.

Simple Types (cont)

- `l : LOGICAL` has values `FALSE`, `UNKNOWN`, and `TRUE`, with `FALSE < UNKNOWN < TRUE`.
- `b : BOOLEAN` is a 'subtype' of `LOGICAL` having values of `FALSE` and `TRUE` only.

Comparisons on Booleans and Logicals can be performed (e.g `=, <, "<=", "<>"`, etc.)

Other operations include `NOT`, `AND`, `OR`, `XOR`.

Simple Types (cont)

- `s : STRING` - a sequence of characters
- `bin : BINARY` - a sequence of bits (0s and 1s)

These may be dynamic or fixed with a maximum size. For example `STRING(6) FIXED`.

These types may be concatenated and compared, and subsets addressed via indexing. For example

```
s1 : STRING := 's';
s2 : STRING := 'its';
.....
s1 := s1 + s2;
IF s1[2:3] = 'it' THEN ...
```

Aggregations

General form is `AGGR [L:H] OF ⋯` where L and H are the Low and High bounds respectively (`H >= L`), and containing N elements. Bags, Lists and Sets may have an indefinite high bound denoted by '?' character.

ARRAY

Ordered collection of elements. `N = (H-L+1)`.

BAG

Unordered collection with possibly duplicate elements. `L <= N <= H " where " L >= 0`.

LIST

Ordered collection with possibly duplicate elements. `L <= N <= H " where " L >= 0`.

SET

Unordered collection with no duplicate elements. `L <= N <= H " where " L >= 0`.

> **NOTE** `LIST [L:H] OF UNIQUE ⋯` is used for an ordered collection with no duplicates.

Types

User defined extensions to the simple types and aggregations.

Defined

A 'renaming' of a simple type or aggregation.
`TYPE volume = REAL; END_TYPE;`

Select

A selection among some types.
`TYPE choose = SELECT(a,b,c); END_TYPE;`

Enumeration

An ordered set of values represented by names.
`TYPE enum = ENUMERATION OF (up, down);`
`END_TYPE;`

TYPE Examples

```
TYPE things = SET [1:?] OF
              LIST [1:?] OF thing;
END_TYPE;

TYPE date = ARRAY [1:3] OF INTEGER;
END_TYPE;

TYPE gender = ENUMERATION OF
              (male, female);
END_TYPE;

TYPE hair_type = ENUMERATION OF
                 (blonde, black, bald);
END_TYPE;

TYPE choose_thing = SELECT
                    (thing1, thing2);
```

```
END_TYPE;
```

ENTITY

An entity represents an object of interest in the model of the Universe of Discourse.

The characteristics (properties) of an entity are defined in terms of data (attributes) and behaviour (constraints).

An entity may ‘inherit’ properties from another entity.

ENTITY Attributes

Attributes are either *explicit* or *derived*.

```
ENTITY circle;
  center : point;
  radius : length;
DERIVE
  perimeter : length := 2.0*PI*radius;
END_ENTITY;

TYPE length = REAL; END_TYPE;
```

The data for calculating a derived attribute must be accessible from the entity.

ENTITY Constraints

Attribute values within entity instances may be constrained by either uniqueness requirements or by domain rules (WHERE clauses). These apply to *every* instance of the entity.

```
ENTITY circle;
  center : point;
  radius : length;
UNIQUE
  un1 : center, radius;
WHERE
  pos_rad : radius > 0.0;
END_ENTITY;
```

A WHERE (domain) rule fails if it evaluates to `FALSE`.

Example ENTITY

```
ENTITY person;
  first_name : STRING;
  last_name  : STRING;
  nickname   : OPTIONAL STRING;
  ss_no      : INTEGER;
  sex        : gender;
  spouse     : OPTIONAL person;
  children   : SET [0:?] OF person;
UNIQUE
  un1 : ss_no;
WHERE
  w1 : (EXISTS(spouse) AND sex <> spouse.sex)
       OR NOT EXISTS(spouse);
END_ENTITY;
```

Subtyping

Subtypes inherit ther properties of their Supertypes.

```
ENTITY natural_number;
  value : INTEGER;
END_ENTITY;

ENTITY odd_number
  SUBTYPE OF (natural_number);
  ...
END_ENTITY;

ENTITY prime_number
  SUBTYPE OF (natural_number);
  ...
END_ENTITY;
```

FUNCTION Example

Used for constraint definition and for derived attributes.

```
FUNCTION subset(sub,super :
         AGGREGATE OF GENERIC) : BOOLEAN;

  IF (SIZEOF(sub) > SIZEOF(super)) THEN
    RETURN(FALSE);
  END_IF;
```

```
  REPEAT i := 1 TO SIZEOF(sub);
    IF (sub[i] IN super) THEN
      super := super - sub[i];
    ELSE
      RETURN(FALSE);
    END_IF;
  END_REPEAT;
  RETURN(TRUE);

END_FUNCTION;
```

Predefined Functions

- Mathematical (e.g ABS, SIN, SQRT etc)
- Aggregation sizes (e.g LOBOUND, HIBOUND, SIZEOF, LENGTH)
- Number/String conversion (FORMAT, VALUE)
- EXISTS(V) checks for existance of OPTIONAL attribute V.
- NVL(ATTR; SUBS) if ATTR has a value, then ATTR is returned, else SUBS is returned.
- TYPEOF(V) returns the set of types of V.
- USEDIN(T; R) takes an entity T and its role R that it plays in other entities and returns each entity instance that uses T in role R.

Constants

- Some predefined constants (PI, e).
- User-defined constants

```
CONSTANT
  thousand : NUMBER := 1000;
  million  : NUMBER := thousand**2;
  origin   : point := point(0.0, 0.0);
END_CONSTANT;
```

SCHEMA

- A SCHEMA contains the objects, relationships and constraints for a particular domain of interest.
- Schemas provide a mechanism for partitioning the ‘real world’ into relevant domains.
- There must be well defined limits to the domain represented via a Schema --- a single Schema should not be used to describe two different domains of interest.

SCHEMA (cont)

- An EXPRESS model may contain more than one Schema.
- Where multiple Schemas are used there is normally one ‘main’ schema and n ‘subsidiary’ schemas.

```
SCHEMA main;
  REFERENCE FROM sub1 ...
  -- types, entities, rules, etc.
END_SCHEMA;

SCHEMA sub1;
  -- types, entities, rules, etc.
END_SCHEMA;
```